

SellSync

Backend API Documentation

For Frontend Integration

Version 1.0 • 14 March 2026

1. Overview

SellSync is a POS (Point of Sale) analytics platform for retail store owners and managers. The backend is built with Node.js, Express, Prisma ORM, and PostgreSQL. It provides a RESTful API with AI-powered analytics, a conversational business assistant, real-time notifications, and full audit logging.

Base URL

Dev	<code>http://localhost:5000/api</code>
-----	--

Tech Stack

- Runtime: Node.js with ES Modules
- Framework: Express.js
- Database: PostgreSQL via Prisma ORM
- Authentication: JWT (JSON Web Tokens)
- AI: Groq API (Llama-4-scout model) + Whisper for speech-to-text
- Email: Nodemailer
- PDF Export: PDFKit

- Cron Jobs: node-cron (runs daily analytics at midnight)

Authentication

All protected routes require a Bearer token in the Authorization header:

Header	<code>Authorization: Bearer <your_jwt_token></code>
--------	---

There are two user roles:

- MANAGER — Store owner, has access to all management and analytics endpoints
- CASHIER — Store staff, can only create sales

2. Authentication

All auth routes are public unless stated otherwise.

Method	Endpoint	Description
POST	/auth/signup	Register a new manager account
POST	/auth/login	Login and receive JWT token
GET	/auth/verify-email?token=	Verify email after signup
POST	/auth/forgotPassword	Request password reset link
GET	/auth/me	Get current logged-in user (Protected)
PATCH	/auth/update/:id	Update user name and email (Protected)
PATCH	/auth/deactivate	Deactivate own account (Protected)

POST /auth/signup

Access: **PUBLIC**

Request Body:

Body	<code>{ "name": "string", "email": "string", "password": "string (min 8 chars)" }</code>
------	--

Response: Sends verification email. User must verify before logging in.

POST /auth/login

Access: **PUBLIC**

Request Body:

Body	<code>{ "email": "string", "password": "string" }</code>
------	--

Response:

Response	<pre>{ "success": true, "token": "jwt_token", "data": { id, name, email, role } }</pre>
----------	---

3. Store Management

Access: **MANAGER**

Method	Endpoint	Description
POST	/stores/create	Create a new store
GET	/stores	Get all stores owned by manager
GET	/stores/:storeId	Get a single store by ID
PATCH	/stores/:storeId/update	Update store details
PATCH	/stores/:storeId/deactivate	Deactivate store and all its cashiers
PATCH	/stores/deactivate-all	Deactivate all stores
PATCH	/stores/:storeId/reactivate	Reactivate a store and its cashiers
PATCH	/stores/reactivate-all	Reactivate all stores

POST /stores/create

Request Body:

Body	<pre>{ "name": "string", "location": "string", "email": "string", "phone": "string" }</pre>
------	---

GET /stores

Returns array of all stores owned by the authenticated manager, including isActive status.

4. Cashier Management

Access: **MANAGER**

Method	Endpoint	Description
POST	/cashiers/:storeId/create	Create a cashier account (sends verification email)
GET	/cashiers/:storeId	Get all cashiers in a store
GET	/cashiers/:storeId/:id	Get a single cashier by ID
PATCH	/cashiers/:storeId/deactivate/:id	Deactivate a cashier
PATCH	/cashiers/:storeId/reactivate/:id	Reactivate a cashier

POST /cashiers/:storeId/create

Creates a cashier account under the manager's store. Sends an email verification link to the cashier. Cashier cannot log in until they verify their email.

Request Body:

Body	<pre>{ "name": "string", "email": "string", "password": "string (min 8 chars)" }</pre>
------	--

5. Product Management

Access: **MANAGER**

Method	Endpoint	Description
POST	/products/:storeId/create	Create a single product with inventory
POST	/products/:storeId/create-many	Create multiple products at once
GET	/products/:storeId	Get all products (supports search & filter)
GET	/products/:storeId/:id	Get a single product by ID
PATCH	/products/:storeId/:id/update	Update product details
PATCH	/products/:storeId/:id/deactivate	Deactivate a product

POST /products/:storeId/create

Request Body:

Body	<pre>{ "name": "string", "sku": "string", "category": "string", "price": number, "quantity": number, "lowThreshold": number }</pre>
------	---

The lowThreshold triggers a low stock alert when inventory falls to or below this value.

GET /products/:storeId — Query Params

Supports optional filters via query parameters:

Search	<pre>/products/:storeId?search=coca</pre>
Category	<pre>/products/:storeId?category=Beverages</pre>
Status	<pre>/products/:storeId?isActive=true</pre>

6. Transactions

Method	Endpoint	Description
POST	/transactions/create	Create a sale (CASHIER only)
GET	/transactions/:storeId	Get all transactions with pagination (MANAGER)
GET	/transactions/:storeId/:id	Get a single transaction (MANAGER)
GET	/transactions/:storeId/cashier/:id	Get transactions by cashier (MANAGER)
GET	/transactions/:storeId/date-range	Get transactions in date range (MANAGER)
GET	/transactions/:storeId/summary	Get sales summary (MANAGER)
GET	/transactions/:storeId/export/pdf	Export transactions to PDF (MANAGER)

POST /transactions/create

Access: **CASHIER**

Request Body:

Body

```
{ "paymentMethod": "CASH | CARD | TRANSFER", "items": [{  
  "productName": "string", "quantity": number }], "discount":  
  number (optional) }
```

Automatically decrements inventory and triggers low stock notifications if stock falls below threshold after sale.

GET /transactions/:storeId — Pagination

Access: **MANAGER**

Supports pagination via query parameters:

Pagination

```
/transactions/:storeId?page=1&limit=10
```

Response includes: total, page, totalPages, transactions[]

GET /transactions/:storeId/date-range

Access: **MANAGER**

Query Params

```
?startDate=2026-03-01&endDate=2026-03-14
```

GET /transactions/:storeId/export/pdf

Access: **MANAGER**

Returns a downloadable PDF file containing all transactions with itemised details, cashier names, payment methods, and total revenue summary.

7. Notifications

Notifications are created automatically by the system in two scenarios:

- Low stock alert — triggered immediately when a cashier creates a sale that brings inventory below the lowThreshold
- AI daily insight — triggered every midnight by the cron job after analysing store performance

Method	Endpoint	Description
GET	/notifications	Get all notifications for the logged-in user
PATCH	/notifications/:id/read	Mark a notification as read

Response format for GET /notifications:

Response	<pre>{ "success": true, "notifications": [{ id, title, message, read, createdAt }] }</pre>
----------	--

8. AI Analytics & Insights

The AI analytics system runs automatically every midnight via a cron job. It fetches store data, sends it to the Groq AI API, and saves structured insights to the database. Managers can also manually trigger analysis at any time.

How It Works

- Cron job runs at midnight for all active stores
- Fetches: revenue, top products, low stock items, daily sales trend, stockout predictions, sales comparisons
- Sends structured data to Groq llama-4-scout model
- AI returns: summary, type, severity, and 3-5 specific actionable insights
- Saves insight to database and creates notifications for all store users

Method	Endpoint	Description
GET	/ai/:storeId/insight	Get latest AI insight for a store (MANAGER)

GET /ai/:storeId/insight Response

Returns the most recent AI-generated insight:

Response	<pre>{ "success": true, "title": "short title", "summary": "one sentence overview", "insights": [{ "title": "string", "message": "string" }] }</pre>
----------	--

Insight Types

- LOW_STOCK — Inventory running critically low
- STOCKOUT_RISK — Products predicted to run out within 14 days
- SALES_SPIKE — Unusual increase in sales activity
- SLOW_MOVING — Products with low sales velocity
- TREND_CHANGE — Notable change in sales pattern

Severity Levels

- HIGH — Requires immediate action
- MEDIUM — Monitor closely
- LOW — Informational

9. Conversational Business Assistant

The chat endpoint allows managers to ask natural language questions about their store. The backend detects the intent, queries the database for relevant data, and uses the Groq AI to generate a conversational response.

Method	Endpoint	Description
POST	/chat/:storeId	Send a text message to the assistant (MANAGER)
POST	/chat/:storeId/transcribe	Transcribe an audio message to text (MANAGER)

POST /chat/:storeId

Access: **MANAGER**

Request Body:

Body	<pre>{ "message": "how much did we make today?" }</pre>
------	---

Response:

Response	<pre>{ "success": true, "reply": "Today's sales are ₦82,250 from 1 transaction." }</pre>
----------	--

Supported Intents

The assistant understands the following question types:

- SALES_TODAY — Today's revenue and transaction count

- SALES_YESTERDAY — Yesterday's revenue and transaction count
- SALES_WEEK — This week's revenue vs last week
- TOP_PRODUCTS — Best selling products by units sold
- INVENTORY — Products running low on stock
- SALES_GOAL — Current sales goal progress
- CASHIER_PERFORMANCE — Sales performance per cashier
- AI_INSIGHT — Latest AI analysis summary
- GENERAL — Any other question handled by AI

Example Questions

Sales	How much did we make today? / What were yesterday's sales?
Products	What are my top selling products? / Which products are running low?
Staff	Who is my best performing cashier? / How many sales did Test cashier 1 make?
Goals	Are we hitting our weekly target? / What is our sales goal?

POST /chat/:storeId/transcribe

Access: **MANAGER**

Accepts an audio file and returns transcribed text. Uses Groq Whisper (whisper-large-v3) model.

Request: multipart/form-data

Form Field	Key: "audio" Type: File Accepted: .mp3, .wav, .m4a, .webm
-------------------	---

Response:

Response	{ "success": true, "text": "how much did we make today?" }
-----------------	--

Typical frontend flow: record audio → POST to /transcribe → get text → POST text to /chat → display AI reply.

10. Sales Goals

Access: **MANAGER**

Managers can set daily or weekly revenue targets for their stores. These goals are used by the AI assistant to give contextual performance answers.

Method	Endpoint	Description
POST	/salesGoal/:storeId/create	Create a sales goal
GET	/salesGoal/:storeId	Get current sales goal
PATCH	/salesGoal/:storeId/:id/update	Update a sales goal

POST /salesGoal/:storeId/create

Request Body:

Body
<pre>{ "period": "DAILY WEEKLY", "targetAmount": number }</pre>

11. Audit Logs

Access: **MANAGER**

Every important action in the system is automatically logged with the user ID, action type, entity, and timestamp. Supports pagination.

Method	Endpoint	Description
GET	/auditLogs/:storeId	Get all audit logs for a store (with pagination)
GET	/auditLogs/:storeId/:id	Get a single audit log entry

GET /auditLogs/:storeId — Pagination

Query Params
<pre>?page=1&limit=10</pre>

Response includes: total, page, totalPages, auditLogs[]

Logged Actions

- CREATE_SALE — Cashier created a transaction
- CREATE_PRODUCT — Manager added a product
- UPDATE_PRODUCT — Manager edited a product
- DEACTIVATE_PRODUCT — Manager deactivated a product
- CREATE_STORE — Manager created a store
- DEACTIVATE_STORE — Manager deactivated a store
- CREATE_CASHIER — Manager created a cashier account
- DEACTIVATE_CASHIER — Manager deactivated a cashier
- REACTIVATE_CASHIER — Manager reactivated a cashier
- CREATE_SALES_GOAL — Manager set a sales goal
- UPDATE_SALES_GOAL — Manager updated a sales goal
- LOGIN — User logged in
- VIEW_AI_INSIGHT — Manager viewed AI analytics

12. Key Data Models

User

Fields: id, name, email, role (MANAGER | CASHIER), storeId, isActive, isVerified, createdAt

Store

Fields: id, name, location, email, phone, isActive, ownerId, createdAt

Product

Fields: id, name, sku, category, price, isActive, storeId, createdAt

Related: inventory { quantity, lowThreshold }

Transaction

Fields: id, storeId, cashierId, totalAmount, paymentMethod, createdAt

Related: items[] { productId, quantity, price }

Insight

Fields: id, storeId, productId (null for AI insights), type, severity, title, summary, message (JSON string of insights array), createdAt

Notification

Fields: id, userId, title, message, read, createdAt

SalesGoal

Fields: id, storeId, period (DAILY | WEEKLY), targetAmount, createdAt, updatedAt

AuditLog

Fields: id, storeId, userId, action, entity, entityId, details (JSON string), createdAt

13. Error Handling

All endpoints return consistent error responses:

Status Code	Meaning
200	Success — GET requests
201	Created — POST/PATCH requests
400	Bad Request — Missing or invalid fields
401	Unauthorized — Invalid or expired token
403	Forbidden — Email not verified or account deactivated
404	Not Found — Resource does not exist
500	Internal Server Error

14. Important Notes for Frontend

- The AI insight endpoint returns message as a JSON string — parse it with `JSON.parse()` before rendering insights array
- The chat /transcribe endpoint expects multipart/form-data with a file field named 'audio'
- Cashiers only have access to POST /transactions/create and GET /notifications — all other endpoints require MANAGER role
- Low stock notifications are created per-user — every cashier and the store owner receives them
- The daily cron job runs at midnight — if the manager checks insights before midnight on the first day, the endpoint returns 404 'No insights generated yet'
- Pagination defaults: page=1, limit=10 — always handle totalPages in the UI
- Product search is case-insensitive — 'coca' will match 'Coca Cola'
- Store deactivation cascades — deactivating a store automatically deactivates all its cashiers
- PDF export streams directly — set response type to 'blob' when calling from frontend

— End of Documentation —